DATA STRUCTURES HOMEWORK

1. Write a C program to store n elements in an array using dynamic memory allocation and find largest number.

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
   int n, i, largest;
   int *arr;

   printf("Enter number of elements: ");
   scanf("%d", &n);
   arr = (int *)malloc(n * sizeof(int));

   if (arr == NULL)
   {
      printf("Memory allocation failed");
      return 1;
   }

   printf("Enter %d elements:\n", n);

   for (i = 0; i < n; i++)
   {
      scanf("%d", &arr[i]);
   }

   largest = arr[0];

   for (i = 1; i < n; i++)
   {
      if (arr[i] > largest)
      {
         largest = arr[i];
      }
   }

   printf("Largest number = %d\n", largest);
```

```
    free(arr);

    return 0;
}
```

OUTPUT:

```
Enter number of elements: 5
Enter 5 elements:
12
23
34
45
56
Largest number = 56
```

---

2. Print an array in reverse order using pointers.

```
#include <stdio.h>

int main()
{
    int arr[100], n, i;
    int *left, *right;
    int temp;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter the elements:\n");

    for (i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    left = arr;
    right = arr + n - 1;

    while (left < right)
    {
        temp = *left;
        *left = *right;
        *right = temp;
```

```
        left++;
        right--;
    }

    printf("Reversed array:\n");

    for (i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

OUTPUT:

```
Enter number of elements: 5
Enter the elements:
89
78
67
56
45
Reversed array:
45 56 67 78 89
```

---

3. Write a C program to store information of students using structure. Sort them by Name.

```
#include <stdio.h>
#include <string.h>

struct Student
{
    int roll;
    char name[50];
    float marks;
};

int main()
{
    struct Student s[100], temp;
```

```
    int n, i, j;

    printf("Enter number of students: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++)
    {
       printf("\nEnter details of student %d:\n", i + 1);

       printf("Roll number: ");
       scanf("%d", &s[i].roll);

       printf("Name: ");
       scanf("%s", s[i].name);

       printf("Marks: ");
       scanf("%f", &s[i].marks);
    }
    for (i = 0; i < n - 1; i++)
    {
       for (j = i + 1; j < n; j++)
       {
          if (strcmp(s[i].name, s[j].name) > 0)
          {
             temp = s[i];
             s[i] = s[j];
             s[j] = temp;
          }
       }
    }

    printf("\nStudents sorted by name:\n");

    for (i = 0; i < n; i++)
    {
       printf("\nRoll Number: %d", s[i].roll);
       printf("\nName: %s", s[i].name);
       printf("\nMarks: %.2f\n", s[i].marks);
    }

    return 0;
}
```

OUTPUT:

```
Enter number of students: 3

Enter details of student 1:
Roll number: 3
Name: joe
Marks: 95

Enter details of student 2:
Roll number: 4
Name: sri
Marks: 96

Enter details of student 3:
Roll number: 5
Name: akil
Marks: 93

Students sorted by name:

Roll Number: 5
Name: akil
Marks: 93.00

Roll Number: 3
Name: joe
Marks: 95.00

Roll Number: 4
Name: sri
Marks: 96.00
```

4. Write a C program to add 'n' distances given in inch-feet system using structures.

```
#include <stdio.h>

struct Distance
{
   int feet;
   float inches;
};

int main()
```

```
{
   struct Distance d[100], total = {0, 0};
   int n, i;

   printf("Enter number of distances: ");
   scanf("%d", &n);

   for (i = 0; i < n; i++)
   {
      printf("\nEnter distance %d:\n", i + 1);

      printf("Feet: ");
      scanf("%d", &d[i].feet);

      printf("Inches: ");
      scanf("%f", &d[i].inches);

      total.feet += d[i].feet;
      total.inches += d[i].inches;
   }
   total.feet += (int)(total.inches / 12);
   total.inches = (int)total.inches % 12;

   printf("\nTotal distance = %d feet %.0f inches\n",
         total.feet, total.inches);

   return 0;
}
```

OUTPUT:

```
Enter number of distances: 2

Enter distance 1:
Feet: 65
Inches: 12

Enter distance 2:
Feet: 43
Inches: 12

Total distance = 110 feet 0 inches
```

5. Write a C program to add two complex numbers by passing structure to a function.

```
#include <stdio.h>

struct Complex
{
   float real;
   float imag;
};

struct Complex add(struct Complex c1, struct Complex c2)
{
   struct Complex result;

   result.real = c1.real + c2.real;
   result.imag = c1.imag + c2.imag;

   return result;
}

int main()
{
   struct Complex c1, c2, result;

   printf("Enter real and imaginary of 1st complex number: ");
   scanf("%f %f", &c1.real, &c1.imag);

   printf("Enter real and imaginary of 2nd complex number: ");
   scanf("%f %f", &c2.real, &c2.imag);

   result = add(c1, c2);

   printf("\nSum = %.2f + %.2fi\n", result.real, result.imag);

   return 0;
}
```

OUTPUT:

```
Enter real and imaginary of 1st complex number: 2
3
Enter real and imaginary of 2nd complex number: 5
6
```

Sum = 7.00 + 9.00i

6. Write a C program to maintain inventory management system using structure.

```
#include <stdio.h>

struct Product
{
   int id;
   char name[50];
   int quantity;
   float price;
};

int main()
{
   struct Product p[100];
   int n, i;
   float total = 0;

   printf("Enter number of products: ");
   scanf("%d", &n);

   for (i = 0; i < n; i++)
   {
      printf("\nEnter details of product %d:\n", i + 1);

      printf("Product ID: ");
      scanf("%d", &p[i].id);

      printf("Product Name: ");
      scanf("%s", p[i].name);

      printf("Quantity: ");
      scanf("%d", &p[i].quantity);

      printf("Price: ");
      scanf("%f", &p[i].price);
   }

   printf("\n--- Inventory Details ---\n");

   for (i = 0; i < n; i++)
```

```
    {
        printf("\nProduct ID: %d", p[i].id);
        printf("\nProduct Name: %s", p[i].name);
        printf("\nQuantity: %d", p[i].quantity);
        printf("\nPrice: %.2f", p[i].price);
        printf("\nValue: %.2f\n", p[i].quantity * p[i].price);

        total += p[i].quantity * p[i].price;
    }

    printf("\nTotal Inventory Value = %.2f\n", total);

    return 0;
}
```

OUTPUT:

```
Enter number of products: 2

Enter details of product 1:
Product ID: 101
Product Name: mariegold
Quantity: 3
Price: 10

Enter details of product 2:
Product ID: 102
Product Name: milkbikies
Quantity: 5
Price: 30

--- Inventory Details ---

Product ID: 101
Product Name: mariegold
Quantity: 3
Price: 10.00
Value: 30.00

Product ID: 102
Product Name: milkbikies
Quantity: 5
Price: 30.00
Value: 150.00
```

Total Inventory Value = 180.00

7. Write a C program for decimal to binary, octal and hexadecimal conversion.

```
#include <stdio.h>

void binary(int n)
{
   int a[32], i = 0;

   if (n == 0)
   {
      printf("0");
      return;
   }

   while (n > 0)
   {
      a[i] = n % 2;
      n = n / 2;
      i++;
   }

   for (i = i - 1; i >= 0; i--)
   {
      printf("%d", a[i]);
   }
}

void octal(int n)
{
   int a[32], i = 0;

   if (n == 0)
   {
      printf("0");
      return;
   }

   while (n > 0)
   {
      a[i] = n % 8;
      n = n / 8;
```

```
        i++;
    }

    for (i = i - 1; i >= 0; i--)
    {
        printf("%d", a[i]);
    }
}

void hexadecimal(int n)
{
    int a[32], i = 0, rem;

    if (n == 0)
    {
        printf("0");
        return;
    }

    while (n > 0)
    {
        rem = n % 16;
        a[i] = rem;
        n = n / 16;
        i++;
    }

    for (i = i - 1; i >= 0; i--)
    {
        if (a[i] < 10)
            printf("%d", a[i]);
        else
            printf("%c", 'A' + (a[i] - 10));
    }
}

int main()
{
    int n;

    printf("Enter a decimal number: ");
    scanf("%d", &n);
```

```
    printf("Binary      : ");
    binary(n);

    printf("\nOctal       : ");
    octal(n);

    printf("\nHexadecimal : ");
    hexadecimal(n);

    return 0;
}
```

OUTPUT:

```
Enter a decimal number: 54
Binary      : 110110
Octal       : 66
Hexadecimal : 36
```

---

8. Write a C program to calculate difference between two time periods.

```
#include <stdio.h>

struct Time
{
    int hours;
    int minutes;
    int seconds;
};

int main()
{
    struct Time t1, t2, diff;
    int seconds1, seconds2, difference;

    printf("Enter first time (hours minutes seconds): ");
    scanf("%d %d %d", &t1.hours, &t1.minutes, &t1.seconds);

    printf("Enter second time (hours minutes seconds): ");
    scanf("%d %d %d", &t2.hours, &t2.minutes, &t2.seconds);

    seconds1 = t1.hours * 3600 + t1.minutes * 60 + t1.seconds;
    seconds2 = t2.hours * 3600 + t2.minutes * 60 + t2.seconds;
```

```
    difference = seconds2 - seconds1;

    if (difference < 0)
        difference = -difference;

    diff.hours = difference / 3600;
    difference = difference % 3600;

    diff.minutes = difference / 60;
    diff.seconds = difference % 60;

    printf("\nTime Difference = %d hours %d minutes %d seconds\n",
           diff.hours, diff.minutes, diff.seconds);

    return 0;
}
```

OUTPUT:

```
Enter first time (hours minutes seconds): 3 40 15
Enter second time (hours minutes seconds): 4
15
56

Time Difference = 0 hours 35 minutes 41 seconds
```

---

9. Write a C program to implement an employee management system using structures.

```
#include <stdio.h>

struct Employee
{
    int id;
    char name[50];
    float salary;
    char department[50];
};

int main()
{
    struct Employee e[100];
    int n, i;
    float total = 0, average;

    printf("Enter number of employees: ");
```

```
    scanf("%d", &n);

    for (i = 0; i < n; i++)
    {
        printf("\nEnter details of employee %d:\n", i + 1);

        printf("Employee ID: ");
        scanf("%d", &e[i].id);

        printf("Name: ");
        scanf("%s", e[i].name);

        printf("Salary: ");
        scanf("%f", &e[i].salary);

        printf("Department: ");
        scanf("%s", e[i].department);

        total += e[i].salary;
    }

    printf("\n--- Employee Details ---\n");

    for (i = 0; i < n; i++)
    {
        printf("\nEmployee ID: %d", e[i].id);
        printf("\nName: %s", e[i].name);
        printf("\nSalary: %.2f", e[i].salary);
        printf("\nDepartment: %s\n", e[i].department);
    }

    average = total / n;

    printf("\nTotal Salary = %.2f", total);
    printf("\nAverage Salary = %.2f\n", average);

    return 0;
}
```

OUTPUT:

```
Enter number of employees: 3

Enter details of employee 1:
```

```
Employee ID: 101
Name: joe
Salary: 100000
Department: it

Enter details of employee 2:
Employee ID: 102
Name: mani
Salary: 50000
Department: db

Enter details of employee 3:
Employee ID: 103
Name: sri
Salary: 150000
Department: hr

--- Employee Details ---

Employee ID: 101
Name: joe
Salary: 100000.00
Department: it

Employee ID: 102
Name: mani
Salary: 50000.00
Department: db

Employee ID: 103
Name: sri
Salary: 150000.00
Department: hr

Total Salary = 300000.00
Average Salary = 100000.00
```

10.. Write a C program to calculate area and perimeter of a rectangle and triangle using function pointers.

```
#include <stdio.h>
float rectangleArea(float l, float b)
{
   return l * b;
```

```
}

float rectanglePerimeter(float l, float b)
{
   return 2 * (l + b);
}

float triangleArea(float b, float h)
{
   return 0.5 * b * h;
}

float trianglePerimeter(float a, float b, float c)
{
   return a + b + c;
}

int main()
{
   float l, b;
   float side1, side2, side3;
   float (*area)(float, float);
   float (*perimeter)(float, float);

   printf("Enter length and breadth of rectangle: ");
   scanf("%f %f", &l, &b);

   area = rectangleArea;
   perimeter = rectanglePerimeter;

   printf("\nRectangle Area = %.2f", area(l, b));
   printf("\nRectangle Perimeter = %.2f\n", perimeter(l, b));

   printf("\nEnter three sides of triangle: ");
   scanf("%f %f %f", &side1, &side2, &side3);

   area = triangleArea;

   printf("\nEnter base and height of triangle: ");
   scanf("%f %f", &b, &l);

   printf("Triangle Area = %.2f", area(b, l));
   printf("\nTriangle Perimeter = %.2f\n",
```

```
        trianglePerimeter(side1, side2, side3));

   return 0;
}
```

OUTPUT:

```
Enter length and breadth of rectangle: 10 5

Rectangle Area = 50.00
Rectangle Perimeter = 30.00

Enter three sides of triangle: 4
5
6

Enter base and height of triangle: 2
4
Triangle Area = 4.00
Triangle Perimeter = 15.00
```